

第 4 组最终报告

模块：工具注册 + OS Skills → 桌面 AI 助手 miniyu

- 课程：操作系统设计 • Linux 桌面 Agentic OS
- 第 4 组：工具服务层 (UF0² 组件 Tools)
- 报告日期：2026-09-16
- 版本：v1.0 (终稿, 含 09-11 验收通过之后新增的浏览器 Agent 深化工作)

摘要

本组实现了一个从“工具注册 + OS Skills”逐步生长的桌面 AI 助手 miniyu，把操作系统能力以 MCP 风格的工具注册表 + 技能服务层形式暴露给大语言模型 (Agent)，让 Agent 既能管理文件系统，也能像人一样操作真实软件（桌面 QQ 收发消息、驱动浏览器访问网页、读取屏幕与本地图片），并全程用客观验证代替假设——每一次外发或交互动作都以截图、剪贴板读回、OCR 核对门、DOM 快照或 IMAP 回读作为证据链，而非只看一次函数返回就宣称成功。

项目分两个阶段推进。第一阶段（09-11 已通过最终验收）：从零搭建 65 个工具（含 15 个浏览器工具）+ 26 个技能的 Agentic OS 工具服务层，覆盖文件系统、应用操作、浏览器 CDP、屏幕视觉、邮件、联网搜索、编程辅助，并补齐安全审计（含危险指令硬拦截、三档授权）与跨平台（Windows / Linux）交付。第二阶段（09-11 之后的附加深入工作）：把“LLM 驱动浏览器”从视觉猜点按升级为全结构化寻址的确定性执行，通过 P2 元素定位可靠性、P2-5 语义等待、P2-6 Agent 集成层与 E2E Benchmark 封板，将一个真实的“豆包网页对话”任务从 38 次 LLM 回退、79 万 token 收敛到确定性 6 次 LLM、8.4 万 token；再以 跨页面四类 Benchmark (A1 / A2 / C / D) 验证这套能力不是某一网站的巧合，并在真实公共网页（MDN）暴露、定位并修复了结构化读取契约缺口（read-contract v2），最终 A1/A2/C/D 四矩阵全 PASS、零回归。

最终交付：65 个注册工具、26 个技能、705 项单元测试（38 个测试文件）全部通过，多次真实 LLM 端到端与真机（QQ / Edge / 豆包 / 阿里云百炼）验证，Windows / Linux 双平台可移植。

一、项目定位与背景

课程要求实现“在 Linux 桌面环境上的 Agentic OS”（Ubuntu 22.04 + GNOME + Python 3.10，见设计任务书）。整个系统按数据流分层：用户自然语言 → AI Shell / HostAgent → 系统协调与安全 → 任务规划 → 执行（优先 GUI、可回退工具）→ 工具服务层 → 能力层，最后由审计与 RAG 记录执行轨迹。

第 4 组承担工具服务层：它是对接“执行层与上层 Agent”和“真实操作系统能力”的关键一层，需要满足五项设计目标。

设计目标	度量标准
统一性	上层只面对一份工具清单、一个调用入口、一种返回结构
可审计性	每次调用可统计次数与结果、操作可回溯
安全性	危险 / 外发操作可被签名与风险分级约束
可联调性	提供与正式版同构的 Mock，其它组可离线开发
可移植性	Windows / Linux 双平台同一套代码

随着第一阶段的推进，本组职责实际扩展为完整的桌面 AI 助手 miniyu（工具服务层 → Agent 编排层 → 用户交互层 CLI + Web 全链路），并把第 5 组职责（系统协调、安全、RAG、审计）一并落地，保证整体项目闭环。

二、总体成果（关键数字）

指标	数值	说明
注册工具总数	65	文件系统 / 应用操作 / 浏览器（15） / 系统管理 / 编程开发
技能总数	26	文件整理 / 查找索引 / 回收备份 / 系统应用 / 通信联网（Agent 白名单 5）
单元测试	705 个全部通过	38 个测试文件，覆盖架构 / 安全 / 浏览器 / 邮件 / 视觉 / 授权等全模块
决策记录 ADR	11 篇	docs/adr/0001~0011，记录工具契约与关键重构决策
Mock 对等	65 工具 / 26 技能	带 [MOCK] 标记，供跨组联调与 CI 离网运行
真机验证	QQ / Edge / 豆包 / 阿里云百炼 / 邮件	桌面 QQ 收发、浏览器端到端、真实 LLM 多步工具调用、SMTP+IMAP 双端闭环
跨页面 Benchmark	A1/A2/C/D 四矩阵	静态 / 动态 / 聊天 / 公共网页，全部 PASS、零回归

三、系统架构设计

3.1 分层与数据流

本组在整个 Agentic OS 中的数据流位置如下（对应设计书第 1→5→2→3→4 组，第 5 组协调）：



▼

[第5组] SystemCoordinator + SecuritySandbox → check_json {approved, risk_level}

▼

[第2组] TaskPlanner → plan_json

▼

[第3组] 执行：优先 GUI，可回退到工具

▼

[第4组] ToolRegistry + SkillLibrary → tool_result_json

- └─ MockToolRegistry（同构，供跨组联调）
- └─ 能力层（文件系统 / 进程 / 网络 / 浏览器 / 邮件 / 屏幕）

▼

[第5组] 审计日志 + RAG 执行轨迹

上层只面对一份工具清单、一个 `call` 入口、一种 `tool_result_json` 返回结构，操作系统差异被封在能力层控制器里。

3.2 接口契约

对外统一为设计书规定的 `tool_result_json`：

```
{
  "success": true,
  "result": "...",
  "tool": "organize_downloads",
  "elapsed_ms": 42,
  "ts": "2026-09-09T09:00:00"
}
```

工程扩展在契约基础上追加了 `tool` / `elapsed_ms` / `ts`，供上层审计与 RAG 使用。输入侧提供 `list_available_tools` / `call` / `call_skill` / `get_stats` / `get_tool_signature` / `register` 等入口。接口形式参考 MCP 的 `tools/list`、`tools/call`，并与第 3 组对齐错误码（`TOOL_NOT_FOUND` / `TOOL_ERROR` / `SKILL_NOT_FOUND` / `SKILL_ERROR`）。

3.3 关键设计取舍

- 用系统调用代替脆弱点坐标：凡是操作系统能确定完成的（复制、移动、搜索、磁盘操作）一律工具化，不依赖 GUI 点击——这是本组对“GUI 自动化可靠性”的根本态度。
- Schema 由代码推断：工具参数不透写两份（代码 + Schema），而由函数签名 + docstring 自动生成 JSON Schema，避免两者漂移。
- 工具与技能分层：工具是原子操作，技能是可编排的组合操作，对应设计书“OS Skills 库”。
- 抽象层把操作系统差异封在控制器里：桌面键鼠有 Windows / Linux 双实现；浏览器走 CDP 协议，独立于操作系统，无需双实现。

四、工具服务层：MCP 风格的工具注册与技能编排

4.1 ToolRegistry（65 工具）

核心职责：统一注册、统一调用、Schema 生成、调用统计、工具签名。每个工具注册时自动生成 SHA256 签名，供完整性校验。

功能域分布：

域	代表工具
文件	copy / move / delete / rename、create_file、checksum、compare_files
文件夹	create / delete / list_directory、sort_directory、index_files、manage_archive
文本	read_text_file、write_text_file、edit_file
批量 / 查找	batch_copy / move、find_large / old / empty_files、search_files、search_in_files
磁盘	disk_usage、directory_size
系统 / 进程	current_directory、run_command（沙箱 + 超时）、list_processes、kill_process
网络 / 邮件	network_status、ping_host、send_email
应用 / 屏幕	app_open、app_send_message、read_qq_chat、screen_inspect、activate_window、click_at

4.2 SkillLibrary（26 技能）

把多个原子工具 + 业务逻辑编排成“一步到位”的组合技能。以设计书演示场景 6「整理下载目录」为例：

```
输入: organize_downloads("~/Downloads")
逻辑: 扫描目录所有文件 → 按扩展名映射类别（文档/图片/视频/音频/压缩包/...）
      → 逐类创建子目录 → 移动文件 → 返回"整理完成" + 分类统计
```

查找索引类有 search_file / find_large / recent_files / rebuild_index / query_index；回收备份类有 trash_file / restore_file / empty_trash / safe_delete / backup_file；通信联网类有 app_send_message / read_qq_chat / send_email / browser_search / browser_extract。

4.3 MockToolRegistry

与正式版同构（同名、同参、同返回结构），内存实现不改磁盘，保证跨组联调与测试的安全、可复现。提供 MockToolRegistry / MockSkillLibrary / MockOSServiceAPI 三个类，对应该三层的接口。

五、真实软件自动化：用验证代替假设

这是本组区别于“只在 Mock 上演示”的核心亮点，两个最有代表性的创新点如下。

5.1 应用操作：桌面 QQ 的确定性自动化

问题：桌面 QQ 是自绘 UI，没有给外部的返回值、没有无障碍树，“点到了没、进对话没”全都不可见。

方案（全是可复用的机制）：

1. 剪贴板读回验证——任何操作后用“截图 / 读剪贴板”取证，不靠猜。
2. 探针定位文本框——对一段坐标带逐点「点击 → 粘贴探针串 → Ctrl+A → Ctrl+C → 读剪贴板」，只有真点中文本框、复制真实发生时，剪贴板才含探针串，杜绝假阳性定位。Ctrl+A 与 Ctrl+C 之间再清一次剪贴板，排除“粘贴源本身就带探针”的干扰；命中后清场，避免探针污染后续输入。
3. OCR 标题门——点搜索结果行后裁剪聊天标题横带交给 OCR，标题含目标关键词才放行发送；“识别哪个会话是对的”留给 Agent 层，技能本身保持纯确定性。5 行不命中即报错、零发送。
4. 窗口矩形锚定坐标——`get_window_rect` 取窗口位置后按比例换算区域，分辨率与拖动变化不失效。

一个诚实记录的证伪让这个方案更可信：早期用 `Ctrl+F 搜索 → Enter` 并记录为成功，真机复测发现桌面 QQ 会把焦点落在资料卡而非搜索结果，之前的“成功”是未充分核实的误报。于是重写为「点击 + 探针 + OCR 门」，ADR 0008 完整记录。结论：用验证代替假设；对真实软件做端到端测试，而不是只测 Mock 或只看返回码。

5.2 浏览器结构化控制（CDP）

- 快照索引：`snapshot()` 给可交互元素打标记并返回压缩清单，Agent 按索引操作，规避 CSS 选择器幻觉与大 DOM。
- 中文安全输入：走 CDP `Input.insertText` 直插文本，避免剪贴板 / 按键盲操作。
- 事件驱动等待：轮询替代固定 `sleep`，适配 SPA 异步渲染。

5.3 浏览器视觉定位增强（DOM-SoM-坐标混合架构）

三套定位手段按「结构化 → 视觉 → 坐标」降级，收敛到统一 `browser_click(target=...)` 接口（`ref` / `som:N` / CSS 选择器任选），并把“点击即完成”升级为“客观验证”：

- DOM / `ref`（第一级）：`browser_snapshot` 与 `browser_find`（按目标文字 / 名称 / 角色局部搜索，大页面不取整页架构）都返回稳定 `ref`，底层取元素 `id` / 结构路径签名生成，DOM 重排不漂移。
- SoM 视觉（第二级）：`browser_inspect` 截一张“交互元素带编号覆盖层”图，Agent 看图选编号后 `browser_click(target='som:N')`。编号只是视觉标签，真正执行落到实时 DOM 的 `ref` 元素，并用单

一 `InspectSession`（编号→ref 映射 + URL/DOM 指纹失效判定）管理——页面导航 / 大变后旧编号自动失效并抛错，逼 Agent 重新 inspect，杜绝“用过期编号点错”。

- 坐标（第三级）：仅 Canvas / 地图 / 白板这类 DOM + SoM 都不可靠时使用。
- 双重缩放坑：元素坐标一律存 CSS 视口像素，截图标注 / 真实鼠标点击时按 DPR 换算设备像素，高分屏 / Windows 缩放下“截图看到哪就点到哪”。
- 客观 Action Verification: `browser_click / type` 返回 `url_changed / page_changed`（URL + DOM 指纹对比），不做 AI 语义判断，把“动作是否生效”的判断交给 Agent 自身。

真机端到端（真实 Chrome）：`browser_find` 定位必应搜索框 → 按 ref 输入中文 → 结果页 → `browser_find(搜索)` 按 ref 点击 → `browser_inspect` 带编号截图 → `som:1` 真实鼠标点击成功、点击后页面变化自动使会话失效（符合设计）。

六、安全与审计（含第 5 组职责补齐）

6.1 三层安全

危险的 / 外发的操作（删除、清空回收站、QQ 发消息、发邮件、执行命令）都在执行前受约束：

- 工具风险分级：`read_only / low / medium / high`，删除与外发类标注高风险。
- 确认门 + 三档授权：`agent.authorization` 三档——基础档全部高危需确认；高级档仅永久删除文件需确认，其余 HIGH（`run_command`、杀进程、QQ 发送、发邮件）自动放行；全自动档从不确认。Web 顶栏与 CLI 均可切换，切换「全自动」前先弹一次浏览器确认把关。
- Agent 编排层执行引擎：`failsafe`（`call_safely / run_skill_safely`）保持按 HIGH 执行。

6.2 危险指令硬拦截（沙箱守卫）

把确认门升级为：带 `cmd/command` 参数的工具一旦命中 `DANGEROUS_ACTIONS`，绕过确认门直接硬拦截（含 Windows / PowerShell 变体 `stop-computer / remove-item`），杜绝“确认弹窗被误点”的社交工程绕过。真机实测：`rm -rf /`、格式化 C 盘 直接拒绝无确认弹窗；`shutdown` 修复前弹确认窗、修复后直接拒绝。

6.3 Agent 实时安全门

Agent 执行链增加 `_sandbox_guard` 接入 `_execute_one`。高端危险指令无论授权档位，一律硬拦截。

6.4 协调 / RAG / 审计

第 5 组职责（本应由第 5 组承担）在整体项目里由本组统一实现以保闭环：

- SystemCoordinator: 独立 `core/coordinator.py`，按 1→5→2→3→4 数据流编排，审计 + RAG 存储。

- RAGKnowledgeBase: 轻量「词频向量 + 余弦相似度」检索执行轨迹（中文 bigram 分词，零外部依赖，可换 embedding 升级）。
- AuditLog: JSON Lines 可落盘、线程安全、事后查询。
- 降级 Mock: `MockCoordinator` / `MockRAG` / `MockAudit`，基于同构工具 Mock 跑端到端编排。

七、Agent 编排层：可调度、可降级、跨模型

为了让 Agent 从“只依赖云端 API 的演示”变成“断网也能演示、任意模型都能接”的形态，编排层做了以下能力：

1. FailoverClient 自动降级——主 API 超时 / 鉴权失败 / 网络不通时自动切本地 Ollama（纯 CPU 可跑），5 分钟定时重试、恢复后自动升级回主 API。给 LLM 接“备用电源”，现场断网也不至于整个 Agent 瘫掉。
2. 模型热切换——下拉框改成可搜索 Combobox（在线 / 本地分组，点选即切），换模型前自动清理不适配的工具消息。
3. function calling 运行时探测 + 双协议自适应——对不支持 `tools` 参数的模型，自动降级为“文本协议”：工具 JSON Schema 注入 system prompt，模型用 `<tool_call>{...}</tool_call>` 标记输出调用，客户端解析回 ReAct 循环。function calling 从“模型必须有”变成“运行时探测 + 自适应”，任意模型都能接工具链。
4. 联网总开关——`web_search.enabled` 一个开关统一管理所有联网能力（自研 HTTP 搜索与百炼服务端 `enable_search` / `enable_code_interpreter`），关闭即完全离线、如实告知“无法获取”，不编造。
5. Streaming 与思考展示——SSE 全链路推送（reasoning / token / tool_call / confirm / done），前端思考过程实时滚动、完成折叠成“已深度思考”，正文渐进上屏。
6. 越级优化省 token——工具结果超长截断（默认前 4000 字符）+ HTTP 连接池复用（省约 50-200ms / 请求）。

真机验证：阿里云百炼 qwen3.5-plus 以 function-calling 完成“用户一句话 → 多步文件系统工具调用 → 自然语言回复”闭环（证据 `docs/evidence/agent_qwen_live.md`）。

八、浏览器 Agent 深化：从视觉猜点到确定性执行（09-11 之后新增）

第一阶段把浏览器 Agent 从“能操作”做到了“能验证”，但仍存在结构性代价：驱动真实豆包对话时，旧实现用了 38 次有效 LLM 回合、79 万 token，大量消耗来自恢复循环、整页读取、选择器猜测与逐步翻倍的观测截图。第二阶段的主题，就是把这条链路从 recovery-driven（恢复驱动）收敛为 deterministic（确定性执行）。

8.1 P1：观测成本控制（Token 降耗 46%）

两轮“豆包发你好 → 读回复”的真实 E2E 对比：

指标	优化前	P1 后	变化
API 账单消耗	274,606	147,362	-46%
最大并发图片数	6（逐轮翻倍）	2（全程封顶）	止损雪球
末轮图片 token	14,796（约占输入 50%）	4,932	-67%
最大单轮 prompt	29,284	19,664	-33%
功能成功率	成功	成功	持平
账单对账	日志 164k ≠ 账单 274k	日志 147,362 = 账单	一致

根因来自数据而非猜测：单张观测图约 2466 token 且随图片数线性累加。对策是历史图片裁剪（最多保留最近 2 张）+ 成功结构化动作不自动截图（仅 launch / navigate、click 引发页面变化、失败诊断才截）+ System Prompt 要求“结构化读文字”。这让“旧图重发”这一主因被化解，同时成为一个后续所有实验的对照基线。

8.2 P2：元素定位可靠性

把“点击目标”从“想象一个选择器 / 猜一个坐标”升级为稳定可恢复的寻址：

- Target Handle 短句柄层——browser_find 返回短句柄（如 e1 / e2），绑定到当次观察会话，替代长 DOM ref 和用户自猜 CSS；路标 / 重渲染后失效便强制重寻，杜绝“猜 selector”。
- Edit Host 解析——语义 textbox 包装统一一下钻到真实可输入节点，find → type(handle) 稳定成功。
- stale / not_editable → recovery=find——目标失效时自动回到 find 重新定位。

8.3 P2-5：语义等待与增量发现

新增两个高层语义原语 browser_wait_for_change 与 browser_read_latest_reply：

- Anchor 自愈 + semantic_blocks：把聊天 / 结果区识别为语义块（content / control 分类），WAITING → GENERATING → COMPLETED 全过程只发一次，delta 为真增量。
- 回归组件：control / chip 过滤 + dedup，只返回“真正新增”的内容。

真实豆包会话实测：状态迁移完整、delta 非重复。

8.4 P2-6：Agent 集成层 + E2E Benchmark 封板

把 find → type(handle) → wait_for_change() → 读 delta 打包成 Agent 可用的两段高层闭环。三轮豆包 E2E 对比显示架构效果：

维度	旧基线	第一轮	第二轮	第三轮（冻结）
LLM 有效回合	38	8	8	6
本轮消耗 tokens	790,014	115,015	129,188	84,031
Selector 回退	很多	0	0	0
Inspect / SoM	很多	0	0	0
故障恢复	很多	0	0	0
观察 / 验证方式	whole_page_read	whole_page_read	structured_read	structured_read
验证途径	—	—	latest_reply	wait_delta
verified_success	—	✖	✔	✔
benchmark_valid	—	✔	✔	✔

第三轮是冻结基准：Agent 学会利用 `wait_for_change()` 已返回的结构化 `message_delta` 直接作答，省去一次多余 `read`——这是“奖励”而不是“漏用”，验证语义被重新定义为“最终回答是否有可靠工具的结构化证据支撑，而非是否机械调用某个工具”。

同轮还修复了一个记账（accounting）根因：`sum(recorded) ≠ final_cum` 的 10445 差值来自跨运行继承的 LLM 计数器初始值，而非漏记调用。进入 ReAct 前捕获 `initial_cum_total_used` 后，一致性规则改为 `run_total := final_cum - initial_cum`，真实缺口仍判 `invalid`——绝不用基线掩盖真实漏记。

冻结基线（Browser Agent Core v1）：`LLM calls = 6`、`run_total_tokens = 84,031`、`benchmark_valid = true`、`verified_success = true`、`verification_path = wait_delta`，调用链 `find → type(handle, enter) → wait_for_change() → 复用 message_delta → final`，无 `selector fallback`、无 `SoM`、无 `inspect recovery loop`、无重复 `read`。

九、跨页面 Benchmark 与泛化验证（09-11 之后新增，最核心的闭环）

为了让 P2-5 / P2-6 从“豆包特化”变成“通用浏览器 Agent 能力”，本组定义了四类分层 Benchmark 并逐类执行：

- A1 Local Static：本地静态页（打开 → `find` → `click` → `read`）。
- A2 Local Dynamic：本地动态页（点击查询 → 等待结果变化 → 读 `result`，脱离聊天语义，验证 `Anchor / Wait / Delta` 的通用可观测性）。
- C Doubao Chat：真实聊天流式页（`send` → `wait` → `delta` → `answer`），作为参考冻结集。
- D Public Web Smoke：真实公共网页（MDN 文档页）泛化抽查。

四类隔离评价、禁止折总分；D 为抽查项，不进门槛。同时新增 `recovery_depth` 指标——完成任务首次失败后额外产生的 LLM 轮次，比 `action_failures` 更贴近最初 79 万 token 爆炸的根因。

9.1 A1：本地静态页正式 PASS

`find(查看更多) → handle → click(handle) → find(region, 第二段) → handle → read_text(handle)` 全程 `handle` 闭环，`task_success / verified_success / benchmark_valid = true / true / true`，`action_failures=0 / recovery_depth=0 / selector_fallback=0`，`llm_calls=5 / run_total_tokens=52,464`（不到 P1 基线三分之一的成本）。

这条路径还顺带暴露并修复了两个单一根因：`read` 不支持 `handle` 的契约缺口、以及 `driver` 残留浏览器锁 `profile` 导致下次加载旧 DOM 的装置问题——每批只改一层，不通过改 `Core` 或改指标迁就 `benchmark`。

9.2 D：真实公共网页的首个失败样本（get 到真正的工程缺口）

在 MDN 文档页，旧协议首跑失败：`task_success=false`、`recovery=6`、`action_failures=3`、选择器级联失败。根因分析（全程只读、0 token）锁定为结构化读取契约缺口，而非网页复杂：

```
H2#examples_2 (text_len=8, 子: <a>Examples</a>)
└─ SECTION.content-section (text_len=8)      ← 仅含标题锚
    └─ DIV.layout__body (text_len=17675)      ← 示例正文其实在下游容器
```

文档页典型的「标题锚块 + 正文在下游容器」结构下，`read_text(handle)` 只取锚块节点的 `innerText`，取不到下游正文，逼 Agent 去猜 `selector`，触发恢复级联。这不是某一网站的巧合——任何“标题即锚点、正文在 `sibling` 容器”的页面都会触发，属通用的 `observation / read-contract` 缺口。

9.3 read-contract v2：暴露即能力

修复方案不是让 Agent“更努力地猜”，而是把“标题身下有正文”这个结构显式暴露给模型：

- `browser_find` 对标题类目标返回 `related_content {handle, relation, preview, char_count, truncated}`，把「锚点 + 下游正文」绑定为可消费的结构
- `read_text(target=<related_handle>, mode="content")` 用确定性 JS 重推导正文，硬上限 2000 字符、诚实标记截断
- 独立 `handle` 遵循 `stale` 契约；默认行为不变只读单节点（A1 向后兼容）
- `find` 与 `read` 共用单源，不产生分叉

真实浏览器验证（无 LLM）：`find(role=link, Examples) → e2(标题锚) → e2.related_content.handle=e3 → read_text(e3, mode=content)` 读回正文首段——全程无 `selector` 猜测、无 `inspect`、无 `whole-page`。

9.4 四矩阵回归全闭环

模型切到 Qwen3.7-Max 后，`A1 → A2 → C → D` 全矩阵回归：

代号	场景	verified_success	验证路径	recovery_depth	action_failures	判定
A1	Local Static	True	read_targeted	0	0	PASS
A2	Local Dynamic	True	wait_delta	0	0	PASS
C	Doubao Chat	True	wait_delta	0	0	PASS
D	Public MDN	True	read_targeted + related	0	0	PASS

D 复现了 Agent 自主走 `find → related_content.handle → read(mode=content)` 的路径，严格 oracle 「Examples + basic button」双命中；A2 的能力本就正确，仅因 Agent 用 Markdown 表格复述触发了旧 oracle 假阴性，于是把 `task_success` 改为绑定 `wait_delta.message_delta` 三要素判定（纯判定层，不重跑 Agent）；C 前次网络慢 FAIL 对照干净 PASS，证明 benchmark 能区分「真实能力回归」与「外部环境噪声」。

最终闭环：D 失败 → Phase 1 定位缺口 → batch#1 加 related-content → batch#2 自动路由 + 错误语义 → D PASS → A1/A2/C/D 全矩阵零回归，冻结为 P2-8 regression baseline。三条经验沉淀：D 的问题是结构化读取契约不足而非网页复杂；benchmark 本身也是被测系统的一部分（A2 表格假阴性、D 旧版假阳性都源于“结构化证据”与“Agent 排版”不分）；网络伪影单独记录而非归因能力。

十、测试与质量保障

10.1 测试演进

测试数量随工程能力同步增长，且每轮都保证全绿：

```
65 → 81 → 94 → 127 → 217 → 343 → 368 → 370 → 391 → 404 → 415
→ 494 → 539 → 552 → 558 → 583 → 590 → 667 → 672 → 681 → 702 → 705
```

最终 705 项测试、38 个测试文件全部通过。覆盖范围横跨：工具注册与管理、技能编排、Agent ReAct / 安全沙箱硬拦截 / 确认门、Mock 全模块、LLM 客户端 / 离线脑 / Failover 降级、对话记忆、邮件全链路、视觉双通道、授权三档、联网搜索、浏览器 DOM-SoM、语义等待 delta、E2E Benchmark 账目与验证路径、read-contract v2、recovery_depth 等。

10.2 方法论：验证代替假设、隔离归因

本组反复用同一条测试纪律约束自己：

- 验证代替假设——真实软件动作以外置证据（截图 / 剪贴板 / OCR / DOM 快照 / IMAP 回读）确认为准，不只看一次返回码。
- 单一变量、隔离归因——benchmark 里每批只改一层，recovery / prompt / SoM / wait 全冻结；失败先定位单一根因，不通过改 Core 或改指标迁就 benchmark。

3. benchmark 判被测能力产生的证据，而非模型怎么排版答案——A2 的表格假阴性、D 旧版假阳性均由此修正。
4. 能区分真实能力回归与外部噪声——C 的网络慢 FAIL 对照干净 PASS，验证了这套判断的前瞻性。

10.3 Benchmark 记账诚实

账目一致性规则 `run_total := final - initial` 修复后，`benchmark_valid` 成为“功能正确 + 账目有效”的双重保证；第三轮 `initial=0 / final=84,031 / sum=84,031` 完全对账，允许作为冻结基准，而真实漏记仍会被判 `invalid`。

十一、真机验证证据

用例	结果	一句话
QQ 全自动搜索并发送到“我的手机”	通过	探针定位搜索框 → OCR 标题门 → 发送成功，截图取证
QQ 读取会话并自动回复	通过	真实会话读内容 + 自动回复，OCR 复验
Edge / 豆包浏览器对话	通过（免登录可无人值守）	CDP 驱动， <code>contenteditable</code> 输入条真实鼠标点击聚焦 → 插入 → 回车 → 读取回复
浏览器视觉定位（DOM-SOM）	通过（真实 Chrome）	<code>find</code> 拿稳定 <code>ref</code> → 输入 / 点击 → <code>inspect</code> 带编号截图 → <code>som:N</code> 真实鼠标点击
阿里云百炼 <code>qwen3.5-plus</code> 多步文件工具调用	通过	<code>function-calling</code> 驱动“一句话 → 多步工具调用 → 自然语言回复”闭环
邮件 SMTP + IMAP 双端闭环	通过	发送落库 + 收件箱到达核验命中；QQ 改写 <code>Message-ID</code> 等真实坑已逐一修复并回归
Web 六场景实测演示	全部一次跑通	设计书演示场景表逐条执行并经磁盘实体验证
安全机制实测	通过	危险指令硬拦截 + 修复后直接拒绝，对比截图取证

这些证据分散在 `docs/evidence/`、`docs/第4组Web演示记录.md`、`docs/第4组安全机制验证记录.md` 等真机记录中。

十二、跨平台与交付

任务书要求 Linux 桌面，开发却主要在 Windows。逐文件审查后确认架构天然双平台、无需改动：

- 路径 / 编码：全项目相对路径 + `pathlib`；UTF-8 + `safe_print()` 兜住 Windows GBK 终端；LLM 流式按 UTF-8 增量解码。

- 平台自适应层： `app_controller.py` 用抽象基类 + 工厂按 `platform.system()` 分派—— `WindowsAppController` (ctypes/Win32) vs `LinuxAppController` (xdotool/wmctrl/xclip/gnome-screenshot)；浏览器走跨平台 CDP，屏蔽操作系统差异。
- 一键配置 / 启动： `setup_linux.sh` 装依赖 + 生成 config + 检测 Wayland/Xorg； `run_miniyu_web.sh` / `.bat`（纯 ASCII 内容免疫编码问题）。
- 敏感字段不下发： `config.yaml` 个人 key 靠 `.gitignore` + `skip-worktree` 不入库，随代码分发的是空模板 `config.yaml.example`，对方配自己的 key 即可运行。

十三、工程方法论

回顾整个项目，有几条可复用的工程判断：

1. 能力 = 函数暴露 + prompt 守则。给 LLM 的“能力清单”不只是写进 prompt 的话术，而是把更多能力变成白名单函数。QQ 当时只暴露了能发的 `app_send_message`、没有“读”，读能力对它而言不存在；把“读 / 查”做成模型天然可走的 `read_qq_chat` 后，它才会用。提升智能靠补能力函数，而不是要求模型发散。
2. 给 LLM 的联网开关必须稳到单次秒回。旧 `browser_search` 走无头浏览器“打字 + 合成回车”，在必应 / 百度都不触发提交，每次卡 10 秒，模型反复重试把偶发延迟放大成分钟级。改为 HTTP 直接请求后约 0.6 秒返回 5 条——一个不稳定慢函数最伤 Agent 体验。
3. 给 LLM 接“备用电源”。FailoverClient 让主 API 挂掉时自动切本地模型，演示现场断网不掉链子，还能离线自证“Agent 编排”本身不依赖任何一家云厂商。
4. 跨平台是发出去对方配自己 key 就能跑。敏感字段不下发、Windows / Linux 同一套代码、一键脚本 + 空模板，让“别人也能用”成为现实而不是口号。
5. 恢复驱动 → 确定性执行。从 79 万 token 到 8.4 万的真正架构提升，不是省了几个数字，而是把数十轮恢复循环压成单次确定性的浏览器 workflow。

十四、结论与展望

本项目从“工具注册 + OS Skills”出发，长成了一个能管理文件、操作真实软件、驱动浏览器、具备安全审计与跨平台交付的桌面 AI 助手 `miniyu`（65 工具 / 26 技能 / 705 测试全绿），完成了课程要求的工具服务层职责与整体项目闭环。

第二阶段的浏览器 Agent 深化是最有价值的一段：它不仅把一个真实网页任务从 79 万 token 收敛到 8.4 万 token，还用四类跨页面 Benchmark 证明了这套“全结构化寻址”能力是通用而非特化的，并在真实公共网页上完成了一次“失败 → 隔离归因 → 单层修复 → 全矩阵零回归”的完整工程闭环。

可延续的方向：SoM 桌面侧的 `OmniParser` / OCR 扩展、按 `evidence quality` 分级的更多验证路径（`targeted_dom_read` / `accessibility_text` / `vision_observation`）、以及会话级授权的细化。这些

都已留有接口，等待未来接入。

配套文档：`docs/第4组设计文档.md`、`docs/第4组单元测试报告.md`、`docs/第4组联调与集成报告.md`、`docs/第4组调研报告.md`、`docs/第4组接口文档.md`、`docs/第4组交付说明.md`、`docs/P2-7-跨页面-Benchmark-设计.md`、`docs/P2-8-四矩阵回归汇总.md`、`docs/adr/*`。